Name: Ganesh Avirni
Student ID: 23092755
Git URL: https://github.com/avirni5259/Generative-modelling-case-study.git

1. Introduction

This assignment investigates the use of Generative Adversarial Networks (GANs) for synthetic data generation across different data types. A GAN consists of two competing neural networks: a generator, which creates synthetic samples from random noise, and a discriminator, which tries to distinguish real samples from generated ones. Through adversarial training, the generator improves by learning to produce samples that increasingly resemble the real data distribution.

Four GAN-based experiments were completed. First, a simple multilayer perceptron GAN was trained on synthetic two-dimensional data, including a sine wave and a noisy parametric curve. Then, GANs were applied to three real-world tasks: generating BloodMNIST microscope images, producing synthetic CICIDS 2017 network traffic feature vectors, and generating QuickDraw pizza sketches.

Different architectures were used depending on the data type. Fully connected networks were used for the 2D and tabular datasets, while convolutional DCGAN-style models were used for image datasets because convolutional layers are better suited to capturing spatial patterns. The models were evaluated using real-versus-generated visual comparisons, loss curves, PCA plots for tabular data, and simple quantitative distribution comparisons.

2. Part 1: GANs from Scratch on 2D Data

The first experiment reproduced the tutorial GAN using a synthetic sine-wave dataset. The real training samples were two-dimensional points where the first coordinate was randomly sampled and the second coordinate followed a sine function. A simple multilayer perceptron GAN was used because the data consisted of low-dimensional numerical points rather than images.

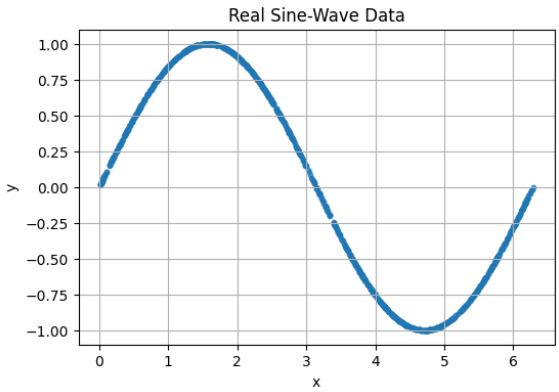

Figure 1: Sine-wave GAN real vs generated samples

The generator took two-dimensional random noise as input and produced two-dimensional fake samples. The discriminator received either real or generated points and predicted whether they were real. Binary cross-entropy loss and Adam optimisers were used for both networks. This matched the tutorial structure and provided a baseline for understanding adversarial training.

After reproducing the sine-wave GAN, a second dataset was created using the noisy parametric curve:

$$y = \sin(2x) + 0.3\cos(5x) + \epsilon$$

This curve was more complex than the original sine wave because it contained multiple oscillations and random noise. The same baseline GAN architecture was first trained on this new distribution. A modified architecture was then tested using larger hidden layers and LeakyReLU activations. This was done to examine whether a slightly deeper model could better capture the more complex shape of the noisy curve.

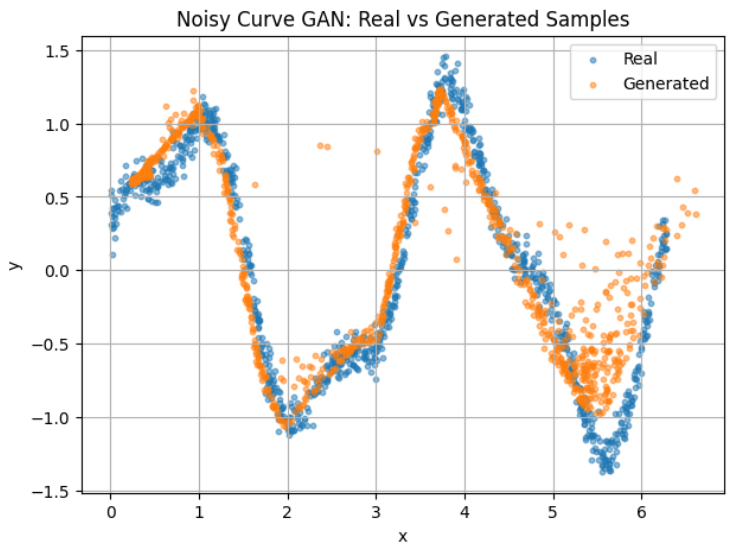

Figure 2: Noisy curve baseline GAN real vs generated samples

The generated samples showed that the GAN was able to learn the general structure of the real 2D distributions. For the sine-wave task, the generated points followed the broad direction of the real curve, although some scatter remained around the target distribution. For the noisy curve, the baseline model captured parts of the pattern but struggled with the sharper variations. The modified architecture produced a better approximation because the larger hidden layers and LeakyReLU activations provided more flexibility for modelling the non-linear distribution.

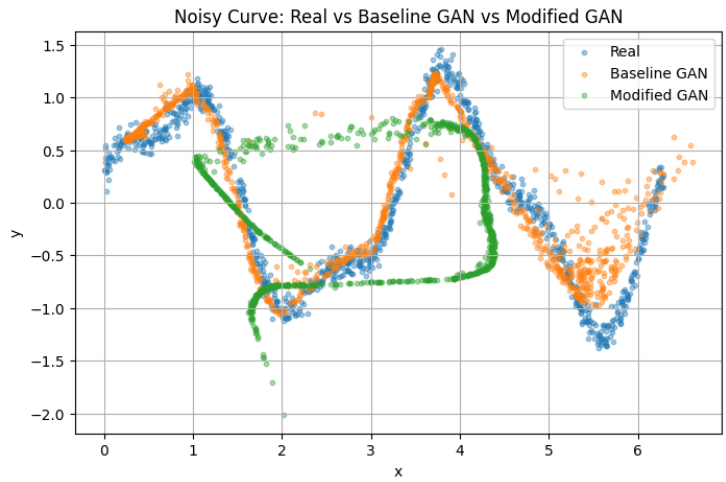

Figure 3: Noisy curve baseline vs modified GAN comparison

Overall, this task showed that GANs can learn simple 2D distributions, but the quality of the generated samples depends strongly on the architecture and stability of training.

3. Part 2.1: BloodMNIST DCGAN

The second experiment applied a DCGAN to the BloodMNIST dataset to generate synthetic blood cell microscope images. BloodMNIST contains small RGB medical images, so a convolutional GAN was used instead of a fully connected network. Convolutional layers are more suitable for image generation because they can learn local spatial features such as cell shape, texture, and colour patterns.

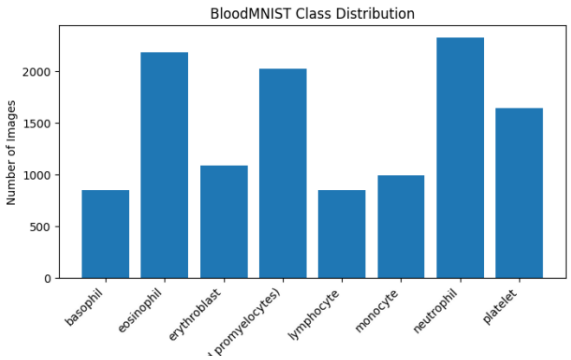

Figure 4: BloodMNIST class distribution

The generator took a random latent vector as input and gradually transformed it into a $3 \times 28 \times 28$image using linear projection, reshaping, transposed convolution layers, batch normalisation, and a final Tanh activation. The discriminator used convolutional layers, LeakyReLU activations, dropout, and a final output logit to classify images as real or generated. BCEWithLogitsLoss was used for training because it combines sigmoid activation and binary cross-entropy in a numerically stable way.

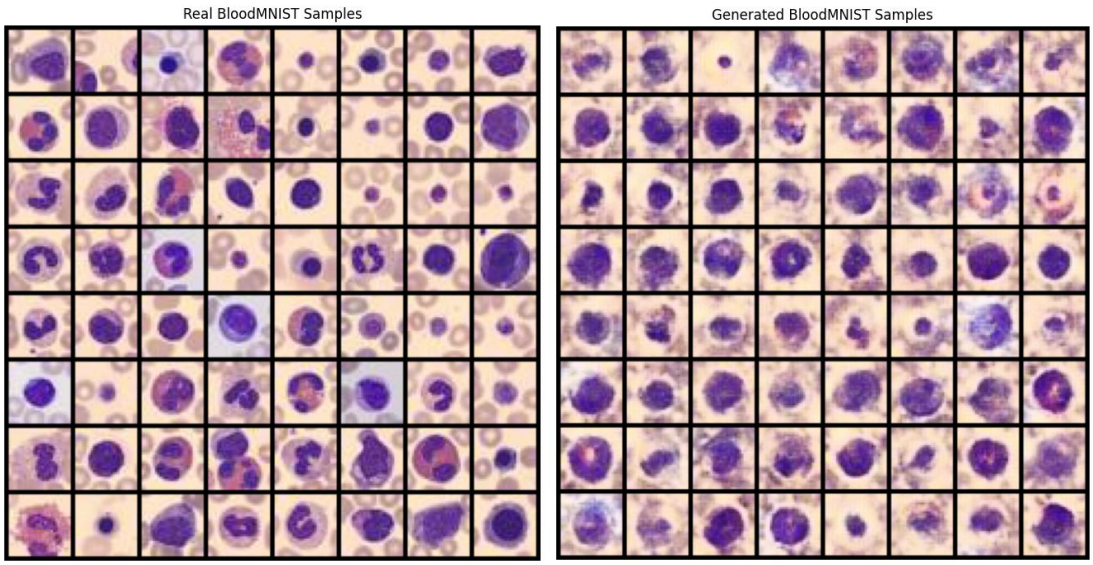

Figure 5: Real and Generated BloodMNIST samples

The class distribution and real image samples were first inspected to understand the dataset. The generated images showed that the DCGAN was able to learn some general properties of BloodMNIST images, especially the circular cell-like shapes and overall colour distribution. However, the fake

images were less detailed than the real images and showed some blurring. This is expected because the model was kept lightweight and trained for a limited number of epochs to remain GPU-friendly.

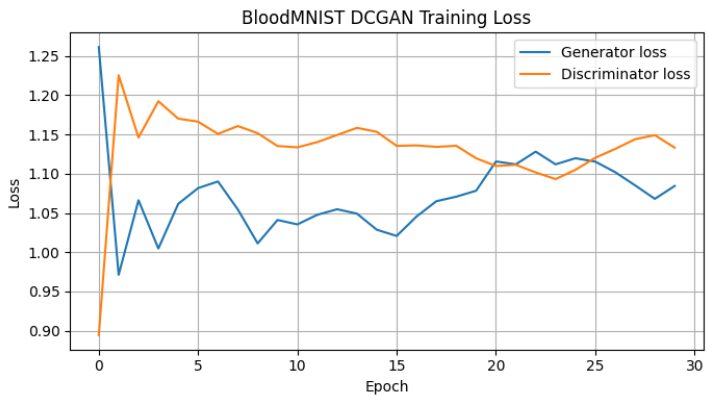

Figure 6: BloodMNIST generator and discriminator loss curve

The loss curves showed the adversarial relationship between the two networks. The discriminator and generator losses did not converge in the same way as standard supervised learning, but fluctuated as both models improved against each other. Overall, the generated samples resembled the real images at a basic visual level, although fine medical details were not fully captured.

4. Part 2.2: CICIDS 2017 Tabular GAN

The third experiment used a GAN to generate synthetic cybersecurity traffic samples from the CICIDS 2017 dataset. Unlike BloodMNIST and QuickDraw, this dataset contains tabular network-flow features rather than images, so a fully connected GAN was used. The Wednesday working-hours file was selected, and the data was filtered to include BENIGN and DoS-related traffic as required.

```
Label distribution:
Label
BENIGN              440031
DoS Hulk            231073
DoS GoldenEye        10293
DoS slowloris         5796
DoS Slowhttptest      5499
Heartbleed              11
Name: count, dtype: int64
```

Figure 7: CICIDS 2017 label distribution

The dataset was cleaned by removing missing and infinite values, then the numerical features were standardised using a StandardScaler. This step was important because GANs are sensitive to feature scale, and network traffic features can vary greatly in magnitude. To keep the training efficient and reduce class imbalance, a limited balanced sample was used from each label.

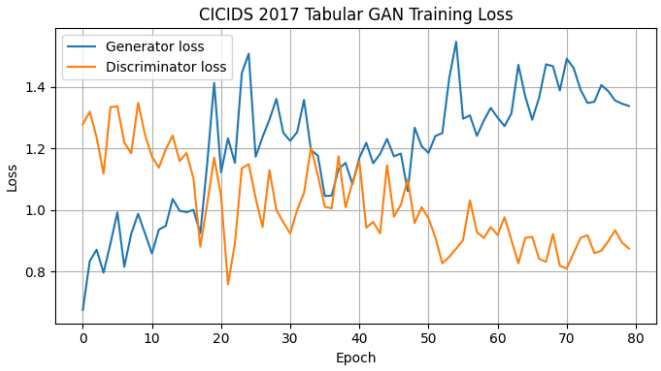

Figure 8: CICIDS GAN loss curve

The generator took a random latent vector and produced synthetic network traffic feature vectors with the same dimensionality as the real data. The discriminator received real or generated feature vectors and predicted whether they came from the real dataset. BCEWithLogitsLoss and Adam optimisers were used for training.

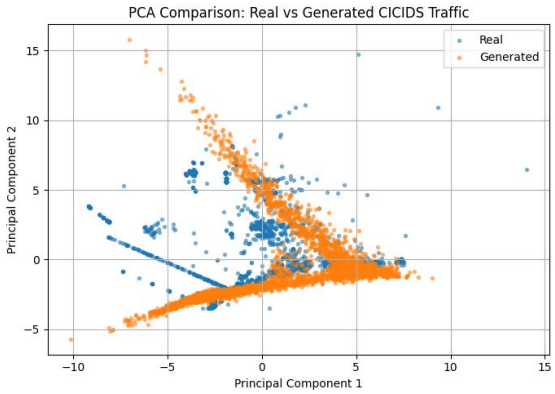

Figure 9: PCA comparison of real vs generated traffic samples

The generated traffic was evaluated using PCA. Real and generated feature vectors were projected into two dimensions and plotted together. The PCA plot showed whether the generated samples occupied a similar region to the real traffic distribution. The results showed that the GAN learned part of the real data structure, although the generated samples did not perfectly overlap with all real clusters. This suggests that the model captured some broad statistical patterns but struggled to reproduce the full complexity of cybersecurity traffic.

Overall, the tabular GAN was useful for generating synthetic network-flow data, but the quality was more difficult to judge than with images. PCA and feature-statistic comparisons were therefore important for evaluating how closely the generated data matched the original distribution.

5. Part 2.3: QuickDraw Pizza DCGAN

The final experiment trained a DCGAN to generate pizza sketches from the QuickDraw dataset. The pizza samples were grayscale 28×28bitmap images, so a convolutional GAN was used. This architecture was suitable because sketch images contain spatial patterns such as outlines, curves, and repeated object features.

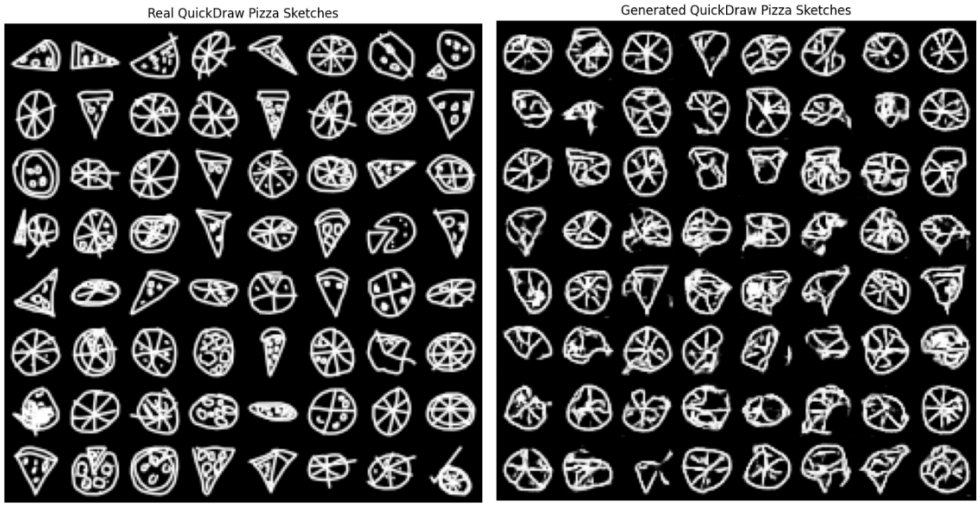

Figure 10: Real and Generated pizza sketches

The generator transformed a random latent vector into a $1 \times 28 \times 28$sketch image using a linear layer, reshaping, transposed convolutions, batch normalisation, and a final Tanh activation. The discriminator used convolutional layers with LeakyReLU activations and dropout to distinguish real sketches from generated ones. The images were normalised to the range $[-1,1]$, matching the generator's Tanh output.

The real pizza sketches showed large variation because QuickDraw drawings are produced by different users with different drawing styles. This made the generation task challenging. The generated samples learned some sketch-like properties, such as dark strokes on a light background and rough circular or triangular pizza-like shapes. However, many generated outputs were still blurry or incomplete, and the model did not always produce clearly recognisable pizzas.

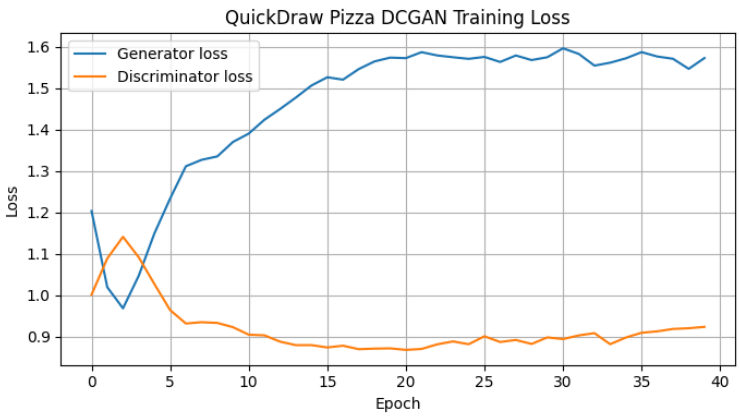

Figure 11: Pizza GAN loss curve

The loss curve showed the unstable but expected adversarial behaviour of GAN training. Although the DCGAN captured the broad visual style of QuickDraw sketches, the results suggest that longer training, a larger model, or class-specific tuning would improve the sharpness and recognisability of the generated sketches.

6. Overall Discussion and Conclusion

Across the four experiments, GANs were able to generate synthetic samples that partially matched the structure of the real datasets. The simplest results were obtained in Part 1, where the data was only two-dimensional and the real distribution could be clearly visualised. The sine-wave and noisy-

curve experiments showed that even a small MLP GAN can learn simple non-linear patterns, although the generated samples still contained scatter and imperfections.

For the image tasks, DCGANs were more suitable because convolutional layers can learn spatial patterns. The BloodMNIST model generated images with cell-like colour and shape patterns, while the QuickDraw model generated sketch-like outputs with dark strokes on a light background. However, both image models showed some blurring and limited detail, mainly because lightweight architectures and limited training epochs were used to keep the experiments computationally manageable.

The CICIDS 2017 task was different because the data consisted of high-dimensional tabular network features. The generated samples were harder to evaluate visually, so PCA and feature-statistic comparisons were used. The GAN captured part of the real traffic distribution, but the PCA plot suggested that generated samples did not fully reproduce all real data clusters.

Overall, the experiments show that GAN performance depends strongly on the data type, architecture, and training stability. MLP GANs are suitable for simple vector data, while DCGANs are more appropriate for images. The results were successful enough to demonstrate synthetic data generation across medicine, cybersecurity, and creative AI, but further improvements could include longer training, hyperparameter tuning, conditional GANs, and stronger quantitative evaluation metrics such as a full FID implementation.